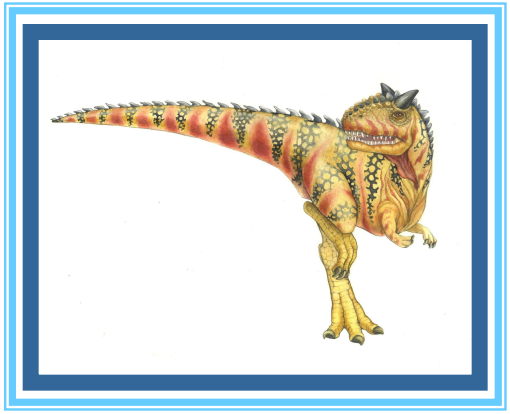
Chapter 2: Operating-System Services



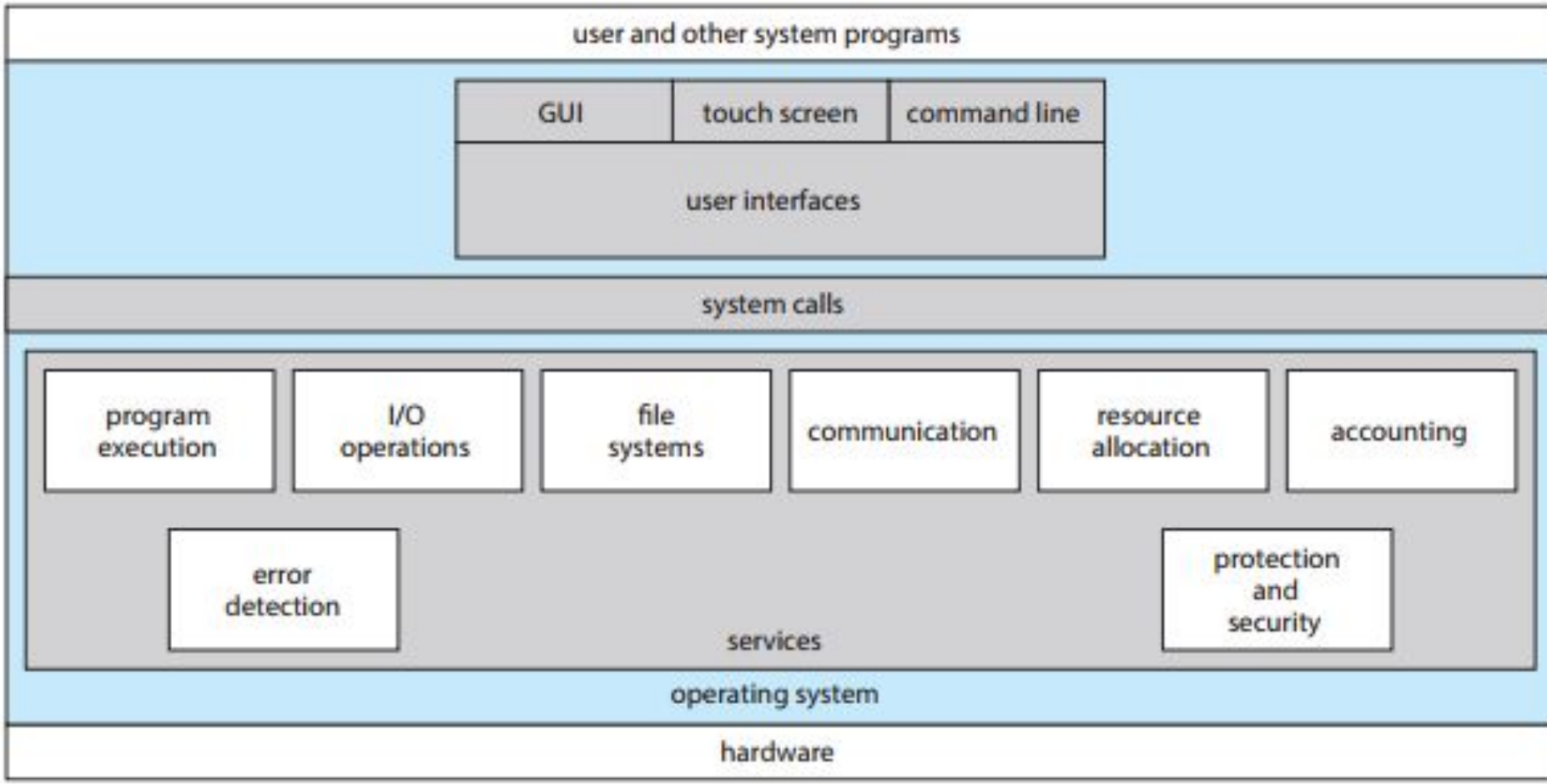
Outline

- Operating System Services
- User and Operating System-Interface
- System Calls
- System Services
- Linkers and Loaders
- Why Applications are Operating System Specific
- Design and Implementation
- Operating System Structure
- Building and Booting an Operating System
- Operating System Debugging

Chapter 2 Operating-System Structures

Objectives

- Identify services provided by an operating system
- Illustrate how system calls are used to provide operating system services
- Compare and contrast monolithic, layered, microkernel, modular, and hybrid strategies for designing operating systems
- Illustrate the process for booting an operating system
- Apply tools for monitoring operating system performance
- Design and implement kernel modules for interacting with a Linux kernel


user and other system programs
GUI
touch screen
command line
user interfaces
system calls
program execution
I/O operations
file systems
communication
resource allocation
accounting
error detection
protection and security
services
operating system
hardware

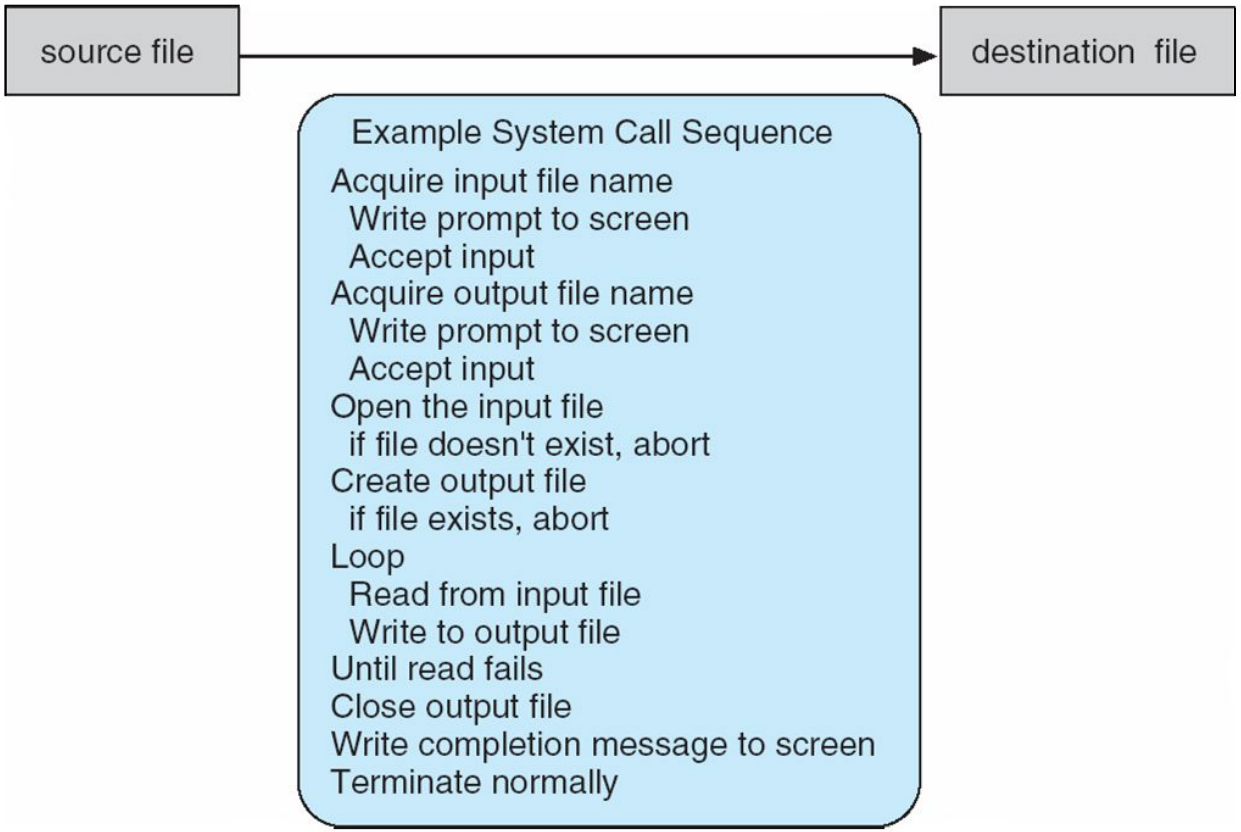
System Calls

- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface** (**API**) rather than direct system call use
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)

Note that the system-call names used throughout this text are generic

Example of System Calls

- System call sequence to copy the contents of one file to another file



```c
#include <stdio.h>

#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[]) {
    // Check if the correct number of arguments is provided
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <source_file> <destination_file>\n", argv[0]);
        exit(EXIT_FAILURE);
    }
    // Open the source file for reading
    int src_fd = open(argv[1], O_RDONLY);
    if (src_fd == -1) {
        perror("Error opening source file");
        exit(EXIT_FAILURE);
    }
    // Open the destination file for writing (create if it doesn't exist, truncate if it does)
    int dest_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dest_fd == -1) {
        perror("Error opening destination file");
        close(src_fd); // Close the source file descriptor
        exit(EXIT_FAILURE);
    }
    // Buffer to hold data read from the source file
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;
```

```
    // Copy data from source to destination
    while ((bytes_read = read(src_fd, buffer, BUFFER_SIZE)) > 0) {
        bytes_written = write(dest_fd, buffer, bytes_read);
        if (bytes_written != bytes_read) {
            perror("Error writing to destination file");
            close(src_fd);
            close(dest_fd);
            exit(EXIT_FAILURE);
        }
    }
    // Check for read errors
    if (bytes_read == -1) {
        perror("Error reading from source file");
        close(src_fd);
        close(dest_fd);
        exit(EXIT_FAILURE);
    }

    // Close file descriptors
    close(src_fd);
    close(dest_fd);
    printf("File copied successfully from %s to %s.\n", argv[1], argv[2]);
    return 0;
}
```

Example of Standard API

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t    read(int fd, void *buf, size_t count)
```

return value — function name — parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

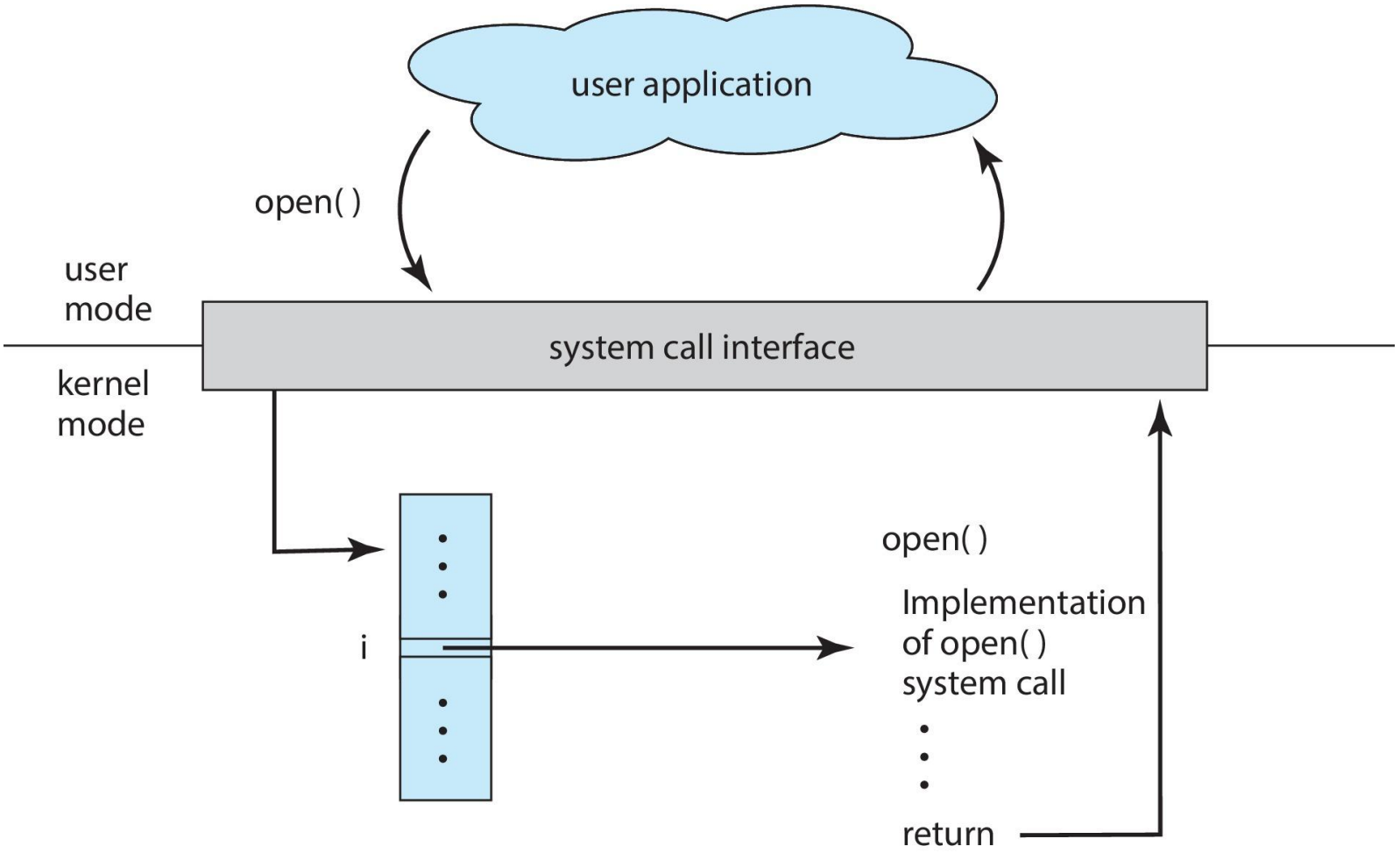
- `int fd`—the file descriptor to be read
- `void *buf`—a buffer into which the data will be read
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns −1.

System Call Implementation

- Typically, a number is associated with each system call
  - **System-call interface** maintains a table indexed according to these numbers
- The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
  - Just needs to obey API and understand what OS will do as a result call
  - Most details of OS interface hidden from programmer by API
    - Managed by run-time support library (set of functions built into libraries included with compiler)

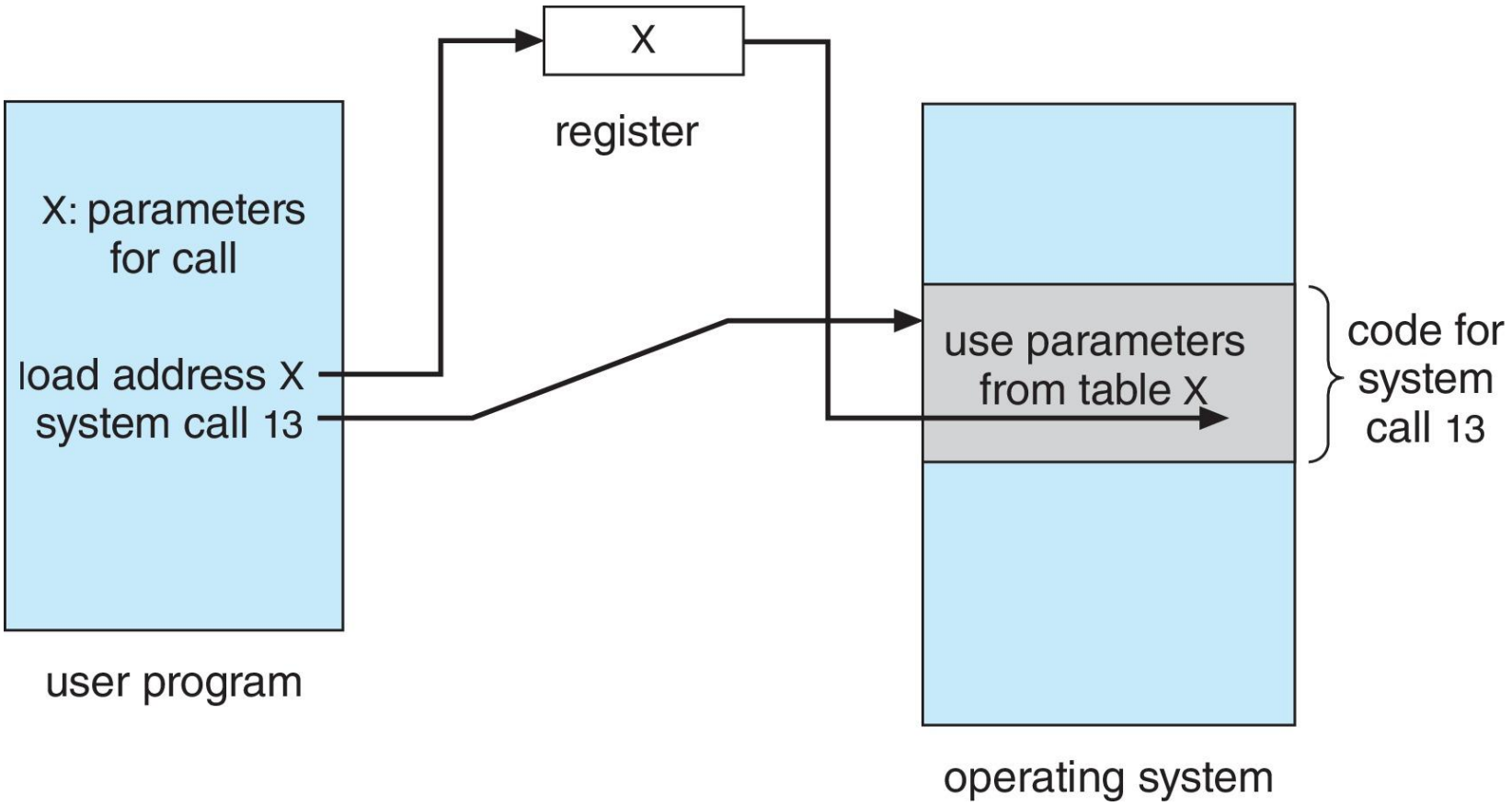
API – System Call – OS Relationship



System Call Parameter Passing

- Often, more information is required than simply identity of desired system call
  - Exact type and amount of information vary according to OS and call
- Three general methods used to pass parameters to the OS
  - Simplest: pass the parameters in registers
    - In some cases, may be more parameters than registers
  - Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
    - This approach taken by Linux and Solaris
  - Parameters placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system
  - Block and stack methods do not limit the number or length of parameters being passed

Parameter Passing via Table



Types of System Calls

- Process control
  - create process, terminate process
  - end, abort
  - load, execute
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory
  - Dump memory if error
  - **Debugger** for determining **bugs, single step** execution
  - **Locks** for managing access to shared data between processes

Types of System Calls (Cont.)

- File management
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes
- Device management
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices

Types of System Calls (Cont.)

- Information maintenance
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- Communications
  - create, delete communication connection
  - send, receive messages if **message passing model** to **host name** or **process name**
    - From **client** to **server**
  - **Shared-memory model** create and gain access to memory regions
  - transfer status information
  - attach and detach remote devices

Types of System Calls (Cont.)

- Protection
  - Control access to resources
  - Get and set permissions
  - Allow and deny user access

Examples of Windows and Unix System Calls

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

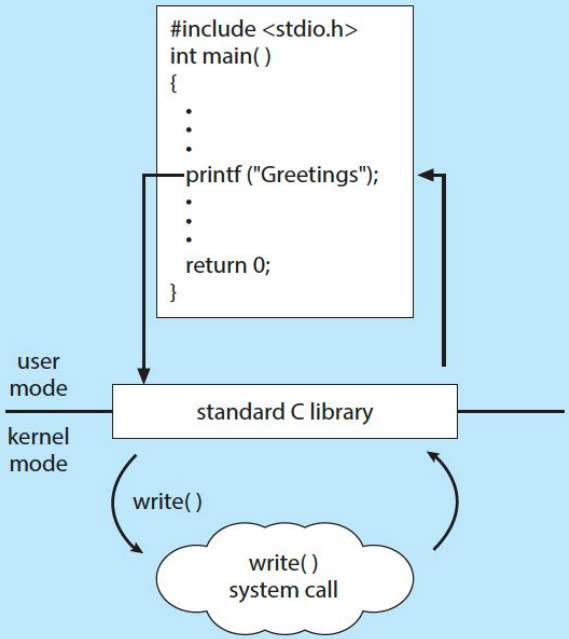
	Windows	Unix
Process control	`CreateProcess()`	`fork()`
	`ExitProcess()`	`exit()`
	`WaitForSingleObject()`	`wait()`
File management	`CreateFile()`	`open()`
	`ReadFile()`	`read()`
	`WriteFile()`	`write()`
	`CloseHandle()`	`close()`
Device management	`SetConsoleMode()`	`ioctl()`
	`ReadConsole()`	`read()`
	`WriteConsole()`	`write()`
Information maintenance	`GetCurrentProcessID()`	`getpid()`
	`SetTimer()`	`alarm()`
	`Sleep()`	`sleep()`
Communications	`CreatePipe()`	`pipe()`
	`CreateFileMapping()`	`shm_open()`
	`MapViewOfFile()`	`mmap()`
Protection	`SetFileSecurity()`	`chmod()`
	`InitializeSecurityDescriptor()`	`umask()`
	`SetSecurityDescriptorGroup()`	`chown()`

Standard C Library Example

- C program invoking printf() library call, which calls write() system call

THE STANDARD C LIBRARY

The standard C library provides a portion of the system-call interface for many versions of UNIX and Linux. As an example, let's assume a C program invokes the `printf()` statement. The C library intercepts this call and invokes the necessary system call (or calls) in the operating system—in this instance, the `write()` system call. The C library takes the value returned by `write()` and passes it back to the user program:



```c
#include <unistd.h>
#include <fcntl.h>
#include <stdio.h>

int main() {
    // Open system call - returns a file descriptor
    int fd = open("test.txt", O_RDONLY);
    if (fd < 0) {
        perror("open failed");
        return 1;
    }
    // Close system call - takes a single integer argument
    if (close(fd) < 0) {
        perror("close failed");
        return 1;
    }
    // Get process ID - no arguments
    pid_t pid = getpid();
    printf("Process ID: %d\n", pid);
    return 0;
}
```

```c
int main() {
    char buffer[1024];
    int fd = open("input.txt", O_RDONLY);
    if (fd < 0) {
        perror("open failed");
        return 1;
    }
    // Read system call - fills a user-space buffer
    ssize_t bytes_read = read(fd, buffer, sizeof(buffer));
    if (bytes_read < 0) {
        perror("read failed");
        close(fd);
        return 1;
    }
    // Write system call - transfers data from a buffer
    ssize_t bytes_written = write(STDOUT_FILENO, buffer, bytes_read);
    if (bytes_written < 0) {
        perror("write failed");
        close(fd);
        return 1;
    }
    close(fd);
    return 0;
}
```

```c
int main() {
    struct stat file_info;

    // stat system call fills a structure with file information
    if (stat("test.txt", &file_info) < 0) {
        perror("stat failed");
        return 1;
    }
    printf("File size: %ld bytes\n", file_info.st_size);
    printf("Last modified: %s", ctime(&file_info.st_mtime));
    // Setting file times using structure
    struct timespec times[2];
    times[0].tv_sec = time(NULL);  // Access time
    times[0].tv_nsec = 0;
    times[1].tv_sec = time(NULL);  // Modify time
    times[1].tv_nsec = 0;
    if (utimensat(AT_FDCWD, "test.txt", times, 0) < 0) {
        perror("utimensat failed");
        return 1;
    }
    return 0;
}
```

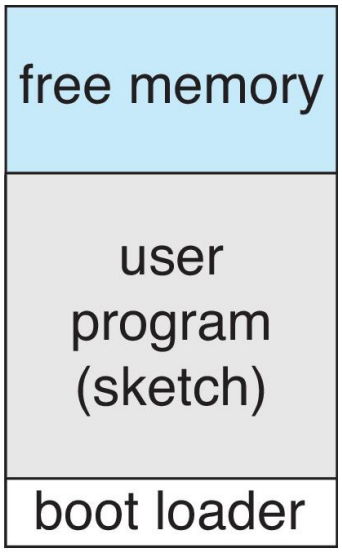
```c
int main() {
    // fcntl can take different numbers and types of arguments
    // depending on the command
    int fd = open("test.txt", O_RDWR);
    if (fd < 0) {
        perror("open failed");
        return 1;
    }
    // Get file flags. Check flag < 0 for error
    int flags = fcntl(fd, F_GETFL);
    // Set file flags (adding O_APPEND)
    if (fcntl(fd, F_SETFL, flags | O_APPEND) < 0) {
        perror("fcntl F_SETFL failed");
        close(fd);
        return 1;
    }
    close(fd);
    return 0;
}
```

Example: Arduino

- Single-tasking
- No operating system
- Programs (sketch) loaded via USB into flash memory
- Single memory space
- Boot loader loads program
- Program exit -> shell reloaded

(a)

At system startup

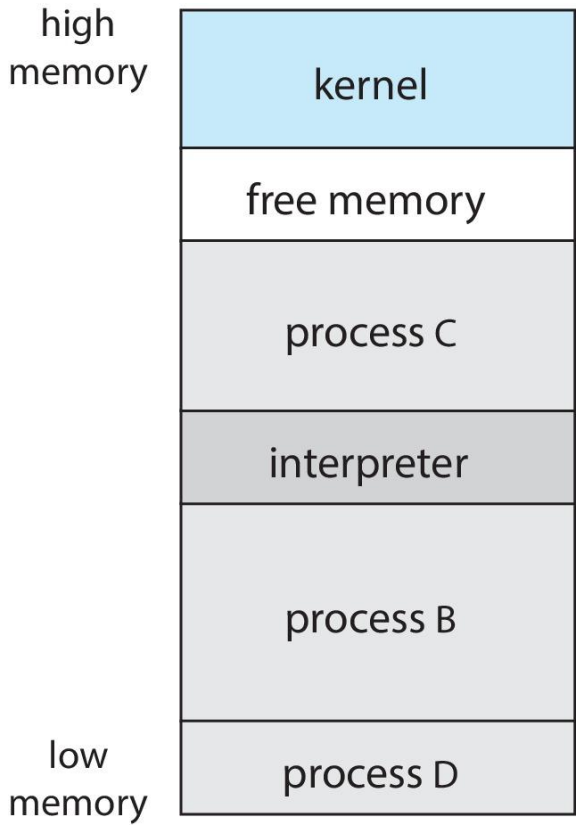


(b)

running a program

Example: FreeBSD

- Unix variant
- Multitasking
- User login -> invoke user's choice of shell
- Shell executes fork() system call to create process
  - Executes exec() to load program into process
  - Shell waits for process to terminate or continues with user commands
- Process exits with:
  - code = 0 – no error
  - code > 0 – error code



System Services

- System programs provide a convenient environment for program development and execution. They can be divided into:
  - File manipulation
  - Status information sometimes stored in a file
  - Programming language support
  - Program loading and execution
  - Communications
  - Background services
  - Application programs
- Most users' view of the operating system is defined by system programs, not the actual system calls

System Services (Cont.)

- Provide a convenient environment for program development and execution
  - Some of them are simply user interfaces to system calls; others are considerably more complex
- **File management** - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories
- **Status information**
  - Some ask the system for info - date, time, amount of available memory, disk space, number of users
  - Others provide detailed performance, logging, and debugging information
  - Typically, these programs format and print the output to the terminal or other output devices
  - Some systems implement a **registry** - used to store and retrieve configuration information

System Services (Cont.)

- **File modification**
  - Text editors to create and modify files
  - Special commands to search contents of files or perform transformations of the text
- **Programming-language support** - Compilers, assemblers, debuggers and interpreters sometimes provided
- **Program loading and execution**- Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
- **Communications** - Provide the mechanism for creating virtual connections among processes, users, and computer systems
  - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another

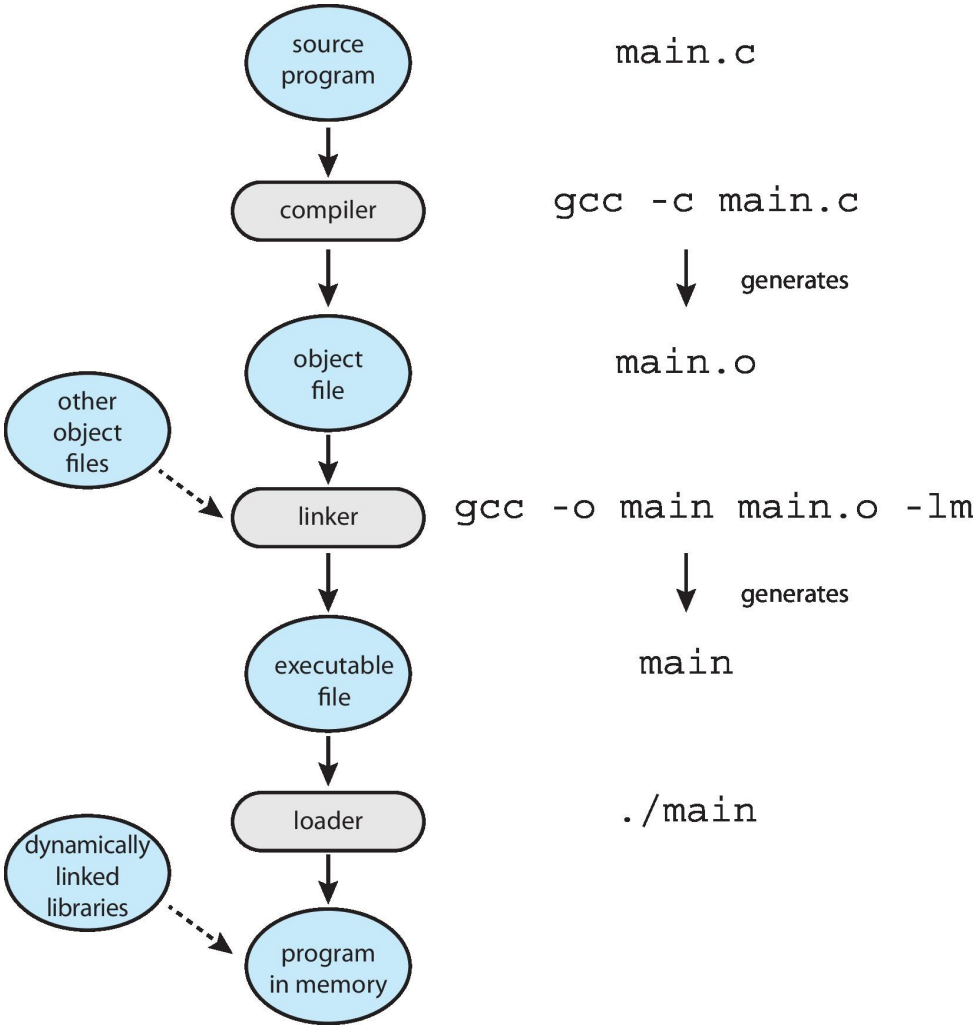
System Services (Cont.)

- **Background Services**
  - Launch at boot time
    - Some for system startup, then terminate
    - Some from system boot to shutdown
  - Provide facilities like disk checking, process scheduling, error logging, printing
  - Run in user context not kernel context
  - Known as **services**, **subsystems**, **daemons**
- **Application programs**
  - Don't pertain to system
  - Run by users
  - Not typically considered part of OS
  - Launched by command line, mouse click, finger poke

Linkers and Loaders

- Source code compiled into object files designed to be loaded into any physical memory location – **relocatable object file**
- **Linker** combines these into single binary **executable** file
  - Also brings in libraries
- Program resides on secondary storage as binary executable
- Must be brought into memory by **loader** to be executed
  - **Relocation** assigns final addresses to program parts and adjusts code and data in program to match those addresses
- Modern general purpose systems don't link libraries into executables
  - Rather, **dynamically linked libraries** (in Windows, **DLLs**) are loaded as needed, shared by all that use the same version of that same library (loaded once)
- Object, executable files have standard formats, so operating system knows how to load and start them

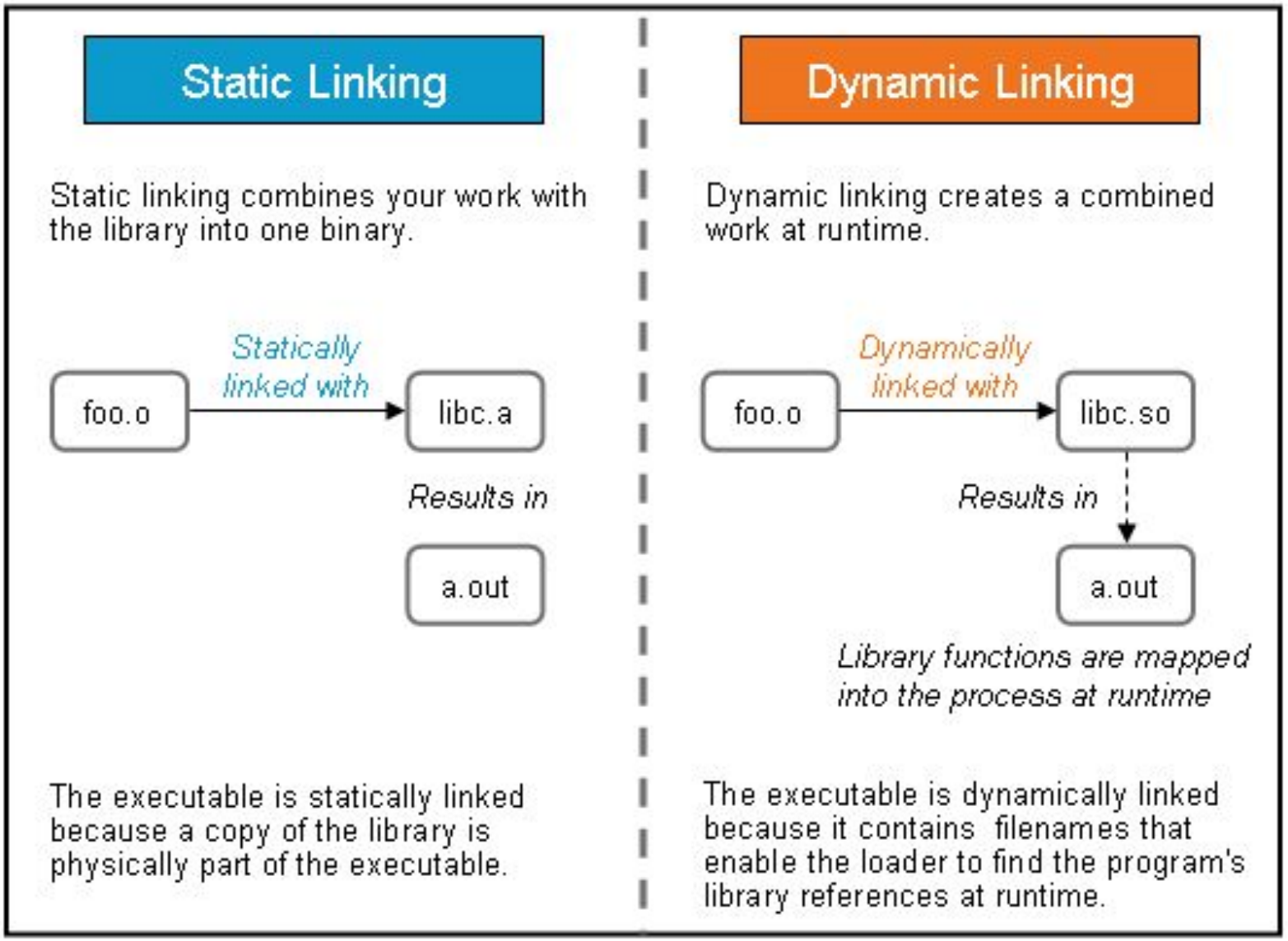
The Role of the Linker and Loader



Linkers and Loaders

Stage	Description
Compilation	Source files are compiled into object files, which are relocatable object files.
Linking	The linker combines relocatable object files into a single binary executable file. It may include other object files or libraries (e.g., C standard library, math library).
Loading	The loader loads the binary executable file into memory, making it eligible to run on a CPU core.
Relocation	Assigns final addresses to program parts and adjusts code and data for correct execution.
Dynamically Linked Libraries (DLLs)	Instead of linking all libraries statically, DLLs are linked at runtime only when needed, saving memory.
Executable Formats	- **ELF (Executable and Linkable Format)**: Used in Linux and UNIX systems. - **PE (Portable Executable Format)**: Used in Windows. - **Mach-O Format**: Used in macOS.

Linkers and Loaders



Why Applications are Operating System Specific

- Apps compiled on one system usually not executable on other operating systems
- Each operating system provides its own unique system calls
  - Own file formats, etc.
- Apps can be multi-operating system
  - Written in interpreted language like Python, Ruby, and interpreter available on multiple operating systems
  - App written in language that includes a VM containing the running app (like Java)
  - Use standard language (like C), compile separately on each operating system to run on each
- **Application Binary Interface** (**ABI**) is architecture equivalent of API, defines how different components of binary code can interface for a given operating system on a given architecture, CPU, etc.

Why Applications Are Operating-System Specific?

Approach	Description	Challenges
Interpreted Languages	Applications written in Python, Ruby, etc., can run on multiple OSs if an interpreter is available.	Slower performance, limited OS features.
Virtual Machine-Based Execution	Languages like Java use a runtime environment (RTE) that allows execution on multiple OSs.	Performance overhead, dependency on RTE.
Compiled and Ported Code	Applications are compiled for each OS separately using standard APIs like POSIX.	Time-consuming, requires recompilation and debugging for each OS.
Binary Format Differences	Each OS has its own executable file format (ELF, PE, Mach-O).	Binary incompatibility across OSs.
CPU Instruction Set Differences	Different CPUs (x86, ARM) have different instruction sets.	Binaries compiled for one architecture won't run on another.
System Call Variations	Different OSs provide different system calls with unique arguments and formats.	Applications need to be adjusted for each OS.
Application Binary Interface (ABI)	Defines binary compatibility for an OS-architecture pair (e.g., ARMv8 ABI).	ABI helps within an OS-architecture pair but does not enable cross-platform execution.

OS Design Goals

Specifying and designing an OS is highly creative task of **software engineering**

User Goals:

- Convenience, ease of use, reliability, safety, and speed.
- Subjective and hard to quantify in design.

System Goals:

- Ease of design, implementation, and maintenance.
- Flexibility, reliability, error-free operation, and efficiency.
- Vague and open to interpretation.

No Unique Solution:

- Requirements vary based on system type (e.g., real-time vs. enterprise).
- Internal structure of different Operating Systems can vary widely
- Examples: Wind River VxWorks (embedded) vs. Windows Server (enterprise).

Policy and Mechanism

Important Principle: Separate of Policy from Mechanism: It allows maximum flexibility if policy decisions are to be changed later.
Example: change 100 to 200

- **Mechanism**: How to do something (e.g., timer for CPU protection).
- **Policy**: What to do (e.g., setting timer duration).
- **Flexibility**: Policies change, mechanisms should remain stable.

Examples:

- **Microkernel OS**: Minimal mechanisms, policies added via modules.
- **Windows/macOS**: Tightly integrated mechanisms and policies for uniformity.
- **Linux**: Open-source, allows modification of mechanisms (e.g., CPU scheduler).

Implementation

- Much variation
  - Early OSes in assembly language
  - Then system programming languages like Algol, PL/1
  - Now C, C++
- Actually usually a mix of languages
  - Lowest levels in assembly
  - Main body in C
  - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- More high-level language easier to **port** to other hardware
  - But slower
- **Emulation** can allow an OS to run on non-native hardware

Implementation

Performance Optimization

1. **Critical Routines**:
   - Interrupt handlers, I/O manager, memory manager, CPU scheduler.
   - Bottlenecks identified and refactored post-implementation.
2. **Key to Performance**:
   - Better data structures and algorithms > assembly-level optimizations.
   - Modern processors handle complex dependencies efficiently.

Implementation

Performance Optimization

1. **Critical Routines**:
   - Interrupt handlers, I/O manager, memory manager, CPU scheduler.
   - Bottlenecks identified and refactored post-implementation.

2. **Key to Performance**:
   - Better data structures and algorithms > assembly-level optimizations.
   - Modern processors handle complex dependencies efficiently.

Operating System Structure

- General-purpose OS is very large program
- Various ways to structure ones
  - Simple structure – MS-DOS
  - More complex – UNIX
  - Layered – an abstraction
  - Microkernel – Mach

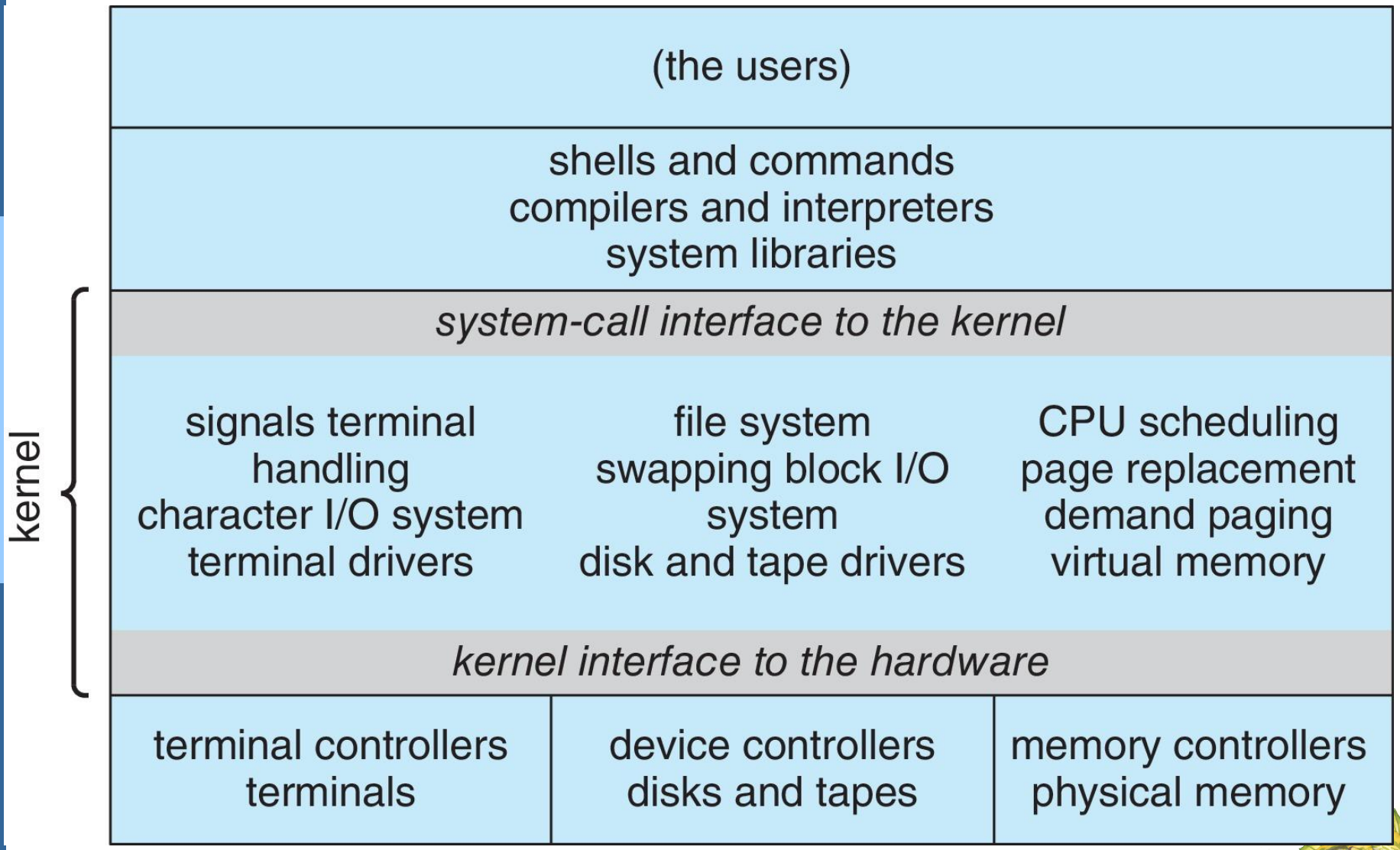
Operating System Structure

Topic	Description
Operating-System Structure	A well-engineered OS is modular, allowing ease of modification and maintenance. The kernel provides core services like file system management, CPU scheduling, and memory management.
Monolithic Structure	A single, static binary where all OS functions exist in one address space. Examples: UNIX, Linux. Pros: Fast performance. Cons: Hard to modify and extend.
Layered Approach	OS is divided into hierarchical layers, with each layer providing specific services to higher layers. Pros: Easier debugging and modification. Cons: Performance overhead due to multiple layers.
Microkernels	Minimalistic kernels with essential OS services. Other services (e.g., file system, device drivers) run in user space and communicate via message passing. Pros: More secure and modular. Cons: Performance overhead due to frequent message exchanges. Examples: Mach (macOS/iOS), QNX.
Modules	A modern approach where OS functions can be loaded dynamically as modules. Offers flexibility and efficiency. Used in Linux and other modern OS designs.

Monolithic Structure – Original UNIX

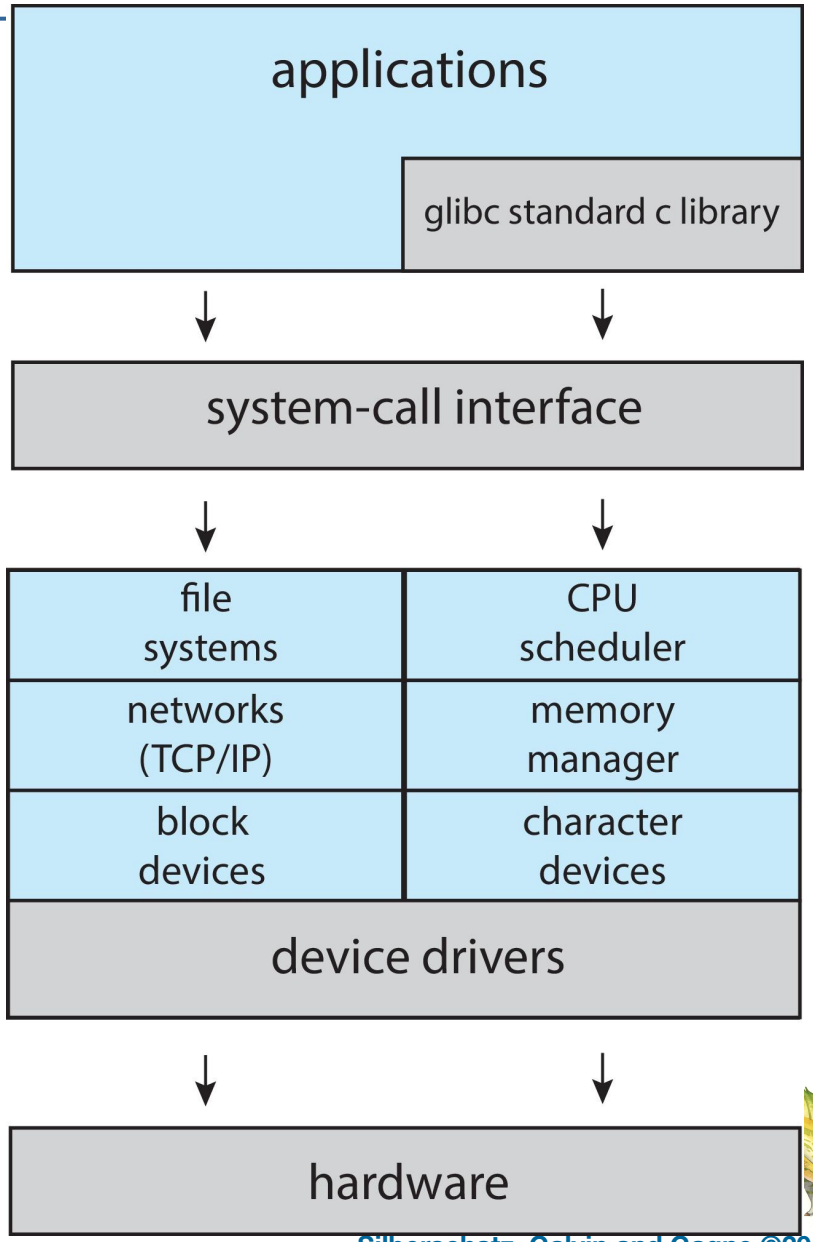
- UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring.
- The UNIX OS consists of two separable parts
  - Systems programs
  - The kernel
    - Consists of everything below the system-call interface and above the physical hardware
    - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level

Traditional UNIX System Structure



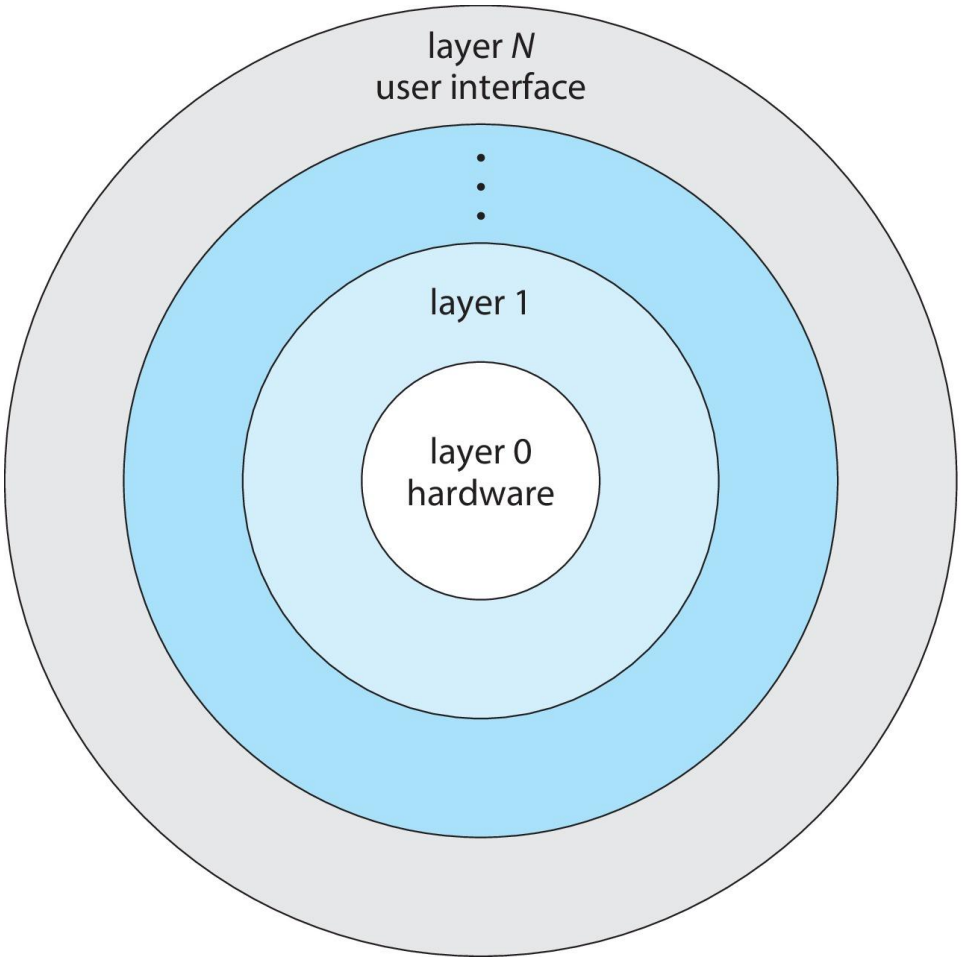
Linux System Structure

Monolithic plus modular design



Layered Approach

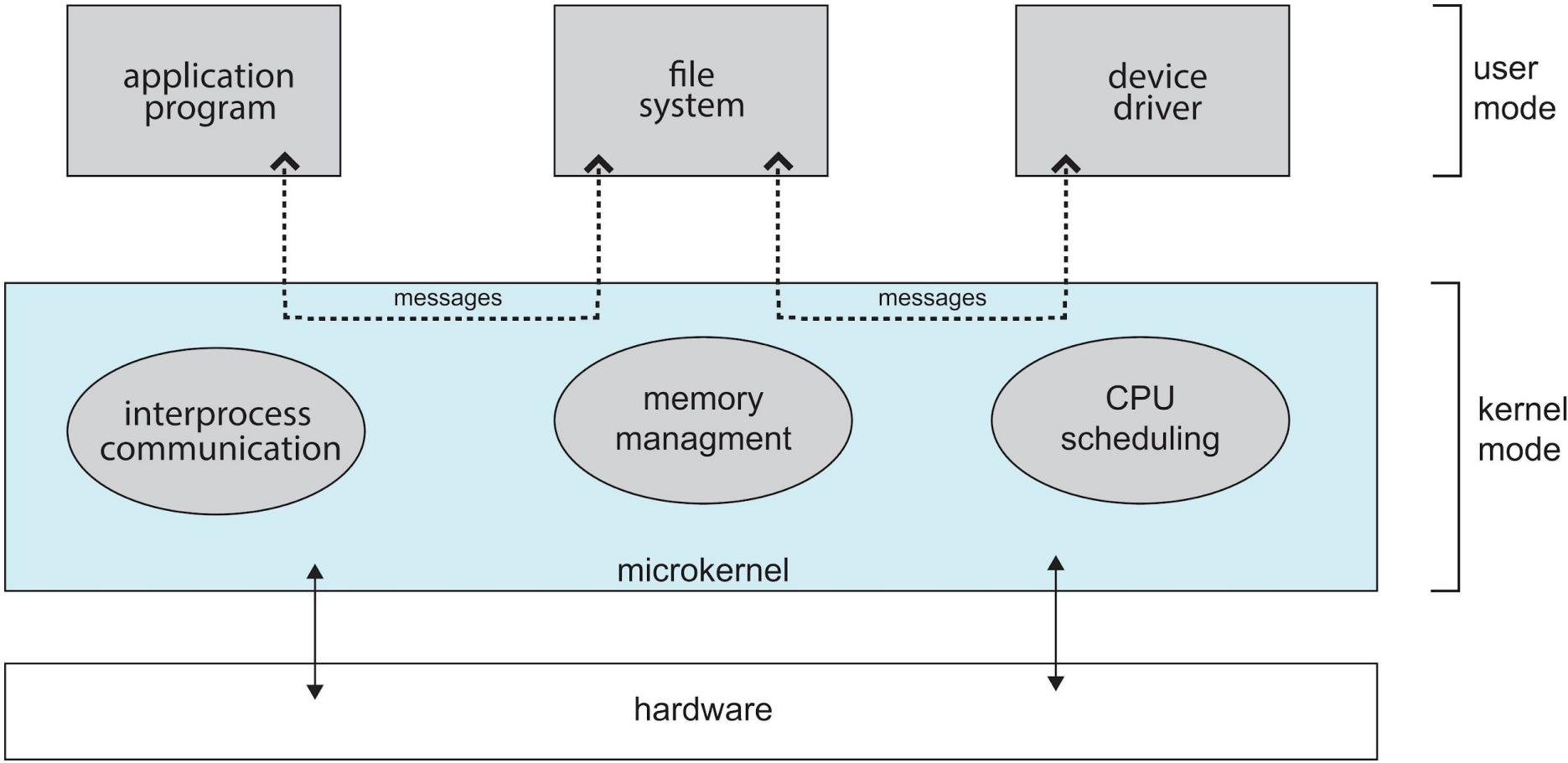
- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers



Microkernels

- Moves as much from the kernel into user space
- **Mach** is an example of **microkernel**
  - Mac OS X kernel (**Darwin**) partly based on Mach
- Communication takes place between user modules using **message passing**
- Benefits:
  - Easier to extend a microkernel
  - Easier to port the operating system to new architectures
  - More reliable (less code is running in kernel mode)
  - More secure
- Detriments:
  - Performance overhead of user space to kernel space communication

Microkernel System Structure



Modules

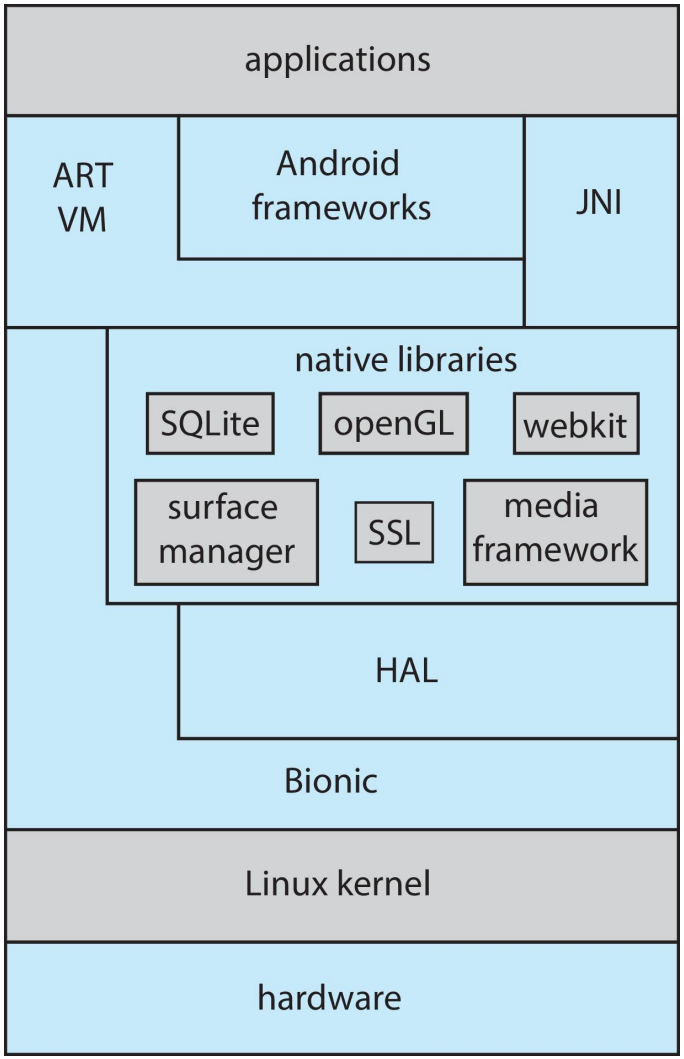
- Many modern operating systems implement **loadable kernel modules** (**LKMs**)
  - Uses object-oriented approach
  - Each core component is separate
  - Each talks to the others over known interfaces
  - Each is loadable as needed within the kernel
- Overall, similar to layers but with more flexible
  - Linux, Solaris, etc.

Feature	Monolithic OS	Layered OS	Microkernel OS	Modular OS
Structure	Single large kernel, all components run in kernel mode.	Divided into layers, each layer interacts with only adjacent layers.	Minimal core kernel; most services run in user space.	Core kernel with dynamically loadable modules.
Performance	High, since everything runs in kernel mode.	Moderate, due to layered interactions.	Lower, due to inter-process communication (IPC) overhead.	High, as only necessary modules are loaded.
Security & Stability	Low, as a bug in any part can crash the entire system.	Higher than monolithic but still vulnerable to failures in lower layers.	Very high, as most services run in user space, isolating failures.	High, as faulty modules can be reloaded without affecting the whole system.
Flexibility & Maintainability	Low, modifying the kernel requires recompilation.	Moderate, changes in a layer require adjusting only adjacent layers.	High, new services can be added without modifying the kernel.	Very high, as modules can be loaded/unloaded dynamically.
Examples	Linux (older versions), MS-DOS, Unix (traditional)	THE OS, MULTICS	Minix, QNX, macOS (partially)	Linux (modern), Windows NT, Solaris

Android

- Developed by Open Handset Alliance (mostly Google)
  - Open Source
- Similar stack to iOS
- Based on Linux kernel but modified
  - Provides process, memory, device-driver management
  - Adds power management
- Runtime environment includes core set of libraries and Dalvik virtual machine
  - Apps developed in Java plus Android API
    - Java class files compiled to Java bytecode then translated to executable thnn runs in Dalvik VM
- Libraries include frameworks for web browser (webkit), database (SQLite), multimedia, smaller libc

Android Architecture



Android Architecture

Layer	Description
1. Linux Kernel	- The foundation of Android. - Handles process management, memory management, security, networking, and hardware abstraction. - Includes device drivers (Wi-Fi, Bluetooth, Display, Camera, etc.).
2. Hardware Abstraction Layer (HAL)	- Acts as an interface between hardware and the higher-level Android system. - Provides standard APIs for hardware-specific implementations like audio, camera, sensors, etc.
3. Native Libraries & Android Runtime (ART)	- Includes essential libraries written in C/C++ (e.g., OpenGL, SQLite, WebKit, SSL). - Android Runtime (ART) executes Android applications using Ahead-of-Time (AOT) compilation.
4. Application Framework	- Provides APIs for higher-level app development. - Key components include: • Activity Manager (manages app lifecycle) • Content Providers (manages shared app data) • Location Manager (handles GPS and location services) • Notification Manager (handles system notifications)
5. Applications	- Topmost layer where user-installed and system apps reside. - Apps use the Application Framework and Android Runtime to interact with the system. - Examples: Dialer, Messages, Browser, Camera, and third-party apps.

Building and Booting an Operating System

- Operating systems generally designed to run on a class of systems with variety of peripherals
- Commonly, operating system already installed on purchased computer
  - But can build and install some other operating systems
  - If generating an operating system from scratch
    - Write the operating system source code
    - Configure the operating system for the system on which it will run
    - Compile the operating system
    - Install the operating system
    - Boot the computer and its new operating system

Building and Booting Linux

- Download Linux source code (http://www.kernel.org)
- Configure kernel via "`make menuconfig`"
- Compile the kernel using "`make`"
  - Produces `vmlinuz`, the kernel image
  - Compile kernel modules via "`make modules`"
  - Install kernel modules into `vmlinuz` via "`make modules_install`"
  - Install new kernel on the system via "`make install`"

Building and Booting Linux

Step No.	Task Description	Command(s)
1	Update and install required dependencies	`sudo apt update && sudo apt install build-essential libncurses-dev bison flex libssl-dev libelf-dev`
2	Download the latest or required kernel source	`wget https://cdn.kernel.org/pub/linux/kernel/vX.Y/linux-X.Y.Z.tar.xz`
3	Extract the downloaded source	`tar -xvf linux-X.Y.Z.tar.xz && cd linux-X.Y.Z`
4	Copy current kernel configuration (optional)	`cp -v /boot/config-$(uname -r) .config`
5	Modify kernel parameters as needed	`make menuconfig` (interactive menu) or manually edit `.config`
6	Clean previous builds (optional)	`make clean && make mrproper`
7	Compile the new kernel	`make -j$(nproc)`

Building and Booting Linux

Step No.	Task Description	Command(s)
8	Install kernel modules	`sudo make modules_install`
9	Install the new kernel	`sudo make install`
10	Update the bootloader (GRUB)	`sudo update-grub`
11	Reboot the system to use the new kernel	`sudo reboot`
12	Verify the new kernel is running	`uname -r`

System Boot

Phase	Description
BIOS/UEFI	The system firmware (BIOS or UEFI) initializes hardware components, performs POST (Power-On Self Test), and identifies bootable devices.
Bootloader (GRUB/LILO)	The bootloader (e.g., GRUB2) loads and presents a boot menu, allowing users to select the OS or kernel parameters. It then loads the selected kernel into memory.
Kernel Initialization	The Linux kernel is decompressed and initialized. It sets up hardware drivers, mounts the initial temporary file system (`initramfs` or `initrd`), and starts essential system services.
init/Systemd Execution	The `init` system (or `systemd` in modern Linux distributions) takes over and initializes system services, including mounting filesystems and starting background daemons.
Runlevel/Target Initialization	The system enters a specified runlevel (SysVinit) or target (systemd) based on configuration, determining which services and processes to start.
User Login Prompt	The system completes booting and presents a login prompt (`tty` for CLI or a graphical display manager for GUI-based login).

System Boot

- When power initialized on system, execution starts at a fixed memory location
- Operating system must be made available to hardware so hardware can start it
  - Small piece of code – **bootstrap loader**, **BIOS**, stored in **ROM** or **EEPROM** locates the kernel, loads it into memory, and starts it
  - Sometimes two-step process where **boot block** at fixed location loaded by ROM code, which loads bootstrap loader from disk
  - Modern systems replace BIOS with **Unified Extensible Firmware Interface** (**UEFI**)
- Common bootstrap loader, **GRUB**, allows selection of kernel from multiple disks, versions, kernel options
- Kernel loads and system is then **running**
- Boot loaders frequently allow various boot states, such as single user mode

End of Chapter 2